LLM-Guided Adaptive Traffic Signal Control with Multi-Intersection Coordination

Yihao Tang
School of Intelligent Systems Engineering
Sun Yat-sen University
Guangzhou, China
tangyh@mail2.sysu.edu.cn

***Abstract*—Traffic signal control in urban road networks remains a challenging optimization problem due to dynamic traffic demand and complex multi-intersection interactions. Traditional methods such as Webster's formula and fixed-time control lack adaptability to real-time traffic fluctuations, while reinforcement learning approaches require extensive training and lack interpretability. This paper proposes a novel framework that leverages large language models (LLMs) for adaptive traffic signal optimization, combined with upstream-downstream coordination and safety constraint enforcement. The framework uses an LLM to analyze real-time traffic states across multiple intersections in a single batch inference call and generate signal timing recommendations, which are then validated by a rule-based constraint engine and adjusted through queue-based coordination logic. Experiments on a 3×2 grid network (6 intersections) using SUMO microscopic simulation demonstrate that the proposed LLM+Coordination strategy achieves 45.6% lower waiting time compared to Webster's method and 39.3% lower than MaxPressure, while matching DQN RL performance without any training. Statistical analysis with paired t-tests confirms significant improvements ($p < 0.001$) over classical baselines. Ablation studies confirm that the LLM provides traffic-aware signal optimization, the constraint engine ensures operational safety, and the coordination engine provides robustness against queue spillover. Scalability experiments on a 4×3 network (12 intersections) show consistent performance gains. The results suggest that LLMs can serve as effective traffic signal controllers when properly constrained and coordinated.**

***Index Terms*—traffic signal control, large language model, multi-intersection coordination, adaptive signal timing, SUMO simulation**

I. Introduction

Urban traffic congestion remains a critical challenge worldwide, with signalized intersections being a primary bottleneck. According to the Texas A&M Transportation Institute, traffic congestion costs the U.S. economy over $87 billion annually. Traditional traffic signal control methods include fixed-time control, actuated control, and optimization-based approaches such as Webster's formula [1]. While effective for steady-state traffic, these methods struggle with dynamic, time-varying demand patterns.

Recent advances in reinforcement learning (RL) have shown promise for adaptive signal control [8], [9]. However, RL approaches suffer from several limitations: (1) they require extensive training on specific network topologies, (2) trained policies often fail to generalize to unseen configurations, (3) they lack interpretability—operators cannot understand why a particular signal timing was chosen, and (4) they require careful reward engineering to avoid degenerate behaviors.

Large language models (LLMs) have demonstrated remarkable reasoning capabilities across diverse domains, including code generation, mathematical problem solving, and planning tasks [23]. A natural question arises: *Can LLMs serve as effective traffic signal controllers?* LLMs offer several potential advantages: they can reason about complex traffic patterns using natural language descriptions, they generalize across different network topologies without retraining, and their decisions can be explained through chain-of-thought reasoning.

This paper makes the following contributions:

- A framework integrating LLM-based decision-making with safety constraint enforcement and multi-intersection coordination (Section III)
- A batch inference approach that processes all intersections in a single LLM call, reducing latency from $O(N)$ to $O(1)$
- Ablation studies isolating the contribution of each component (Section VI)
- Scalability validation on networks of different sizes (Section VII)

II. Related Work

We organize the related literature into four categories: classical signal control, reinforcement learning (RL)-based approaches, LLMs for decision-making and urban computing, and multi-agent coordination.

A. Classical Traffic Signal Control

Traffic signal optimization has a rich history spanning several decades. Webster's formula [1] remains one of the most widely used methods, computing optimal cycle length C_0 and green splits based on critical flow ratios:

$$C_0 = \frac{1.5L + 5}{1 - \sum Y_i} \tag{1}$$

where L is total lost time and Y_i is the critical flow ratio for phase i. Despite its simplicity, this formula provides a strong analytical baseline and is still embedded in commercial signal timing tools. The TRANSYT system [2] extended fixed-time optimization by using platoon dispersion models to coordinate signal offsets across corridors. SCOOT [3] and SCATS [4]

represent the state of the art in deployed adaptive systems, adjusting signal timings incrementally based on real-time detector data. SCOOT operates in over 200 cities worldwide and optimizes cycle length, offsets, and green splits every few seconds. SCATS uses a hierarchical architecture with local controllers selecting from pre-defined plans. Both systems require extensive detector infrastructure and centralized communication. A comprehensive survey of traffic-responsive control can be found in [7].

MaxPressure control [5] takes a fundamentally different approach, selecting signal phases based on queue-length differentials ("pressure") between upstream and downstream links. This method is provably maximally stable under certain network assumptions—meaning it can handle any stabilizable traffic demand—but does not explicitly minimize individual vehicle waiting times. The SOTL (Self-Organizing Traffic Lights) approach [6] demonstrates that simple decentralized rules based on local queue thresholds and neighbor interactions can achieve near-optimal performance in grid networks without any centralized optimization or explicit model of the network topology.

B. RL-Based Traffic Signal Control

Deep reinforcement learning has emerged as a promising paradigm for adaptive signal control, formulating the problem as a Markov decision process (MDP) [32] where the agent observes traffic states and selects signal phases to minimize a reward function (typically total delay or queue length).

Single-intersection methods. IntelliLight [8] introduced a deep Q-network (DQN) with a novel phase-gate mechanism that separates phase selection from duration optimization, enabling more stable training. FRAP [9] proposed a traffic-flow-aware architecture that models competition between traffic movements sharing the same phase, achieving state-of-the-art performance on multiple benchmarks. AttendLight [14] used attention mechanisms to create a universal model that generalizes across intersections with different topologies. Efficient-DQN [13] addressed the challenge of varying traffic demand by incorporating demand-aware features, improving sample efficiency significantly.

Network-level methods. CoLight [10] was among the first to use graph attention networks to capture inter-intersection interactions, enabling coordinated control without explicit communication. PressLight [11] integrated MaxPressure theory with deep RL by using pressure as both the state representation and reward signal, combining theoretical stability guarantees with learning-based optimization. MPLight [12] scaled MaxPressure-based RL to large networks by decomposing the problem into independent agents sharing a common state representation, demonstrating control of up to 1,700 intersections. The multi-agent deep RL framework by Chu et al. [16] trains independent agents per intersection with shared network architectures, achieving scalable coordination through learned communication. Li et al. [15] provided a comprehensive framework for applying deep RL to traffic signals with various state representations and reward designs.

Despite their strong simulation performance, RL approaches face several practical challenges: (1) sample inefficiency, typically requiring millions of training episodes; (2) poor transfer across network topologies—a policy trained on a 4×4 grid may fail on a 3×3 grid; (3) lack of interpretability, making it difficult for traffic operators to understand or trust the decisions; and (4) sensitivity to reward function design, where poorly shaped rewards can lead to degenerate behaviors such as phase starvation.

C. LLMs for Decision-Making and Planning

Large language models [33], [34] have demonstrated remarkable reasoning capabilities that extend beyond text generation. Chain-of-thought (CoT) prompting [17] showed that LLMs can solve complex reasoning tasks by decomposing them into intermediate steps, significantly improving performance on mathematical and logical problems. ReAct [18] extended this by interleaving reasoning traces with action execution, enabling LLMs to interact with external tools and environments. Reflexion [19] further improved LLM agents through verbal self-reflection, allowing agents to learn from past failures without gradient-based training.

In embodied AI and robotics, SayCan [20] grounded LLM planning in physical robot capabilities by scoring actions based on both language likelihood and affordance. Voyager [21] demonstrated open-ended learning in Minecraft using LLMs for skill discovery and code generation. HuggingGPT [22] decomposed complex AI tasks by using an LLM as a controller that selects and orchestrates specialized models. MetaGPT [23] applied LLMs to multi-agent software engineering, assigning different roles (architect, engineer, tester) to LLM agents for collaborative code generation. These works demonstrate that LLMs can serve as effective planners and coordinators in complex multi-step, multi-agent scenarios—properties directly relevant to traffic signal control.

D. LLMs for Transportation and Urban Computing

Several recent works have explored the intersection of LLMs and transportation. TrafficGPT [24] was among the first to integrate LLMs with traffic analysis tools, enabling natural-language interaction with traffic data for tasks such as flow visualization and anomaly detection. UrbanGPT [25] proposed spatio-temporal LLMs for urban computing tasks including traffic prediction and crowd flow forecasting, demonstrating that pre-trained language models can capture complex urban dynamics. CityGPT [26] envisioned LLMs as spatial intelligence engines for urban environments. Surveys by Liang et al. [27] and Fu et al. [28] have catalogued the growing applications of LLMs in transportation, from traffic prediction to autonomous driving scenario generation.

However, none of these prior works have addressed the problem of real-time, multi-intersection traffic signal control using LLMs. TrafficGPT focuses on data analysis rather than control, UrbanGPT targets prediction tasks, and autonomous driving applications address vehicle-level planning rather than infrastructure-level signal optimization. Our work fills this gap

by demonstrating that LLMs can serve as effective traffic signal controllers when combined with safety constraints and coordination mechanisms.

E. Multi-Agent Traffic Control

Coordinating signals across multiple intersections is essential for network-level performance. Early work by Wiering [30] and Abdulhai et al. [31] demonstrated the feasibility of RL-based multi-agent traffic control. El-Tantawy et al. [29] proposed MARLIN-ATSC, one of the most comprehensive multi-agent RL systems for traffic control, demonstrating its effectiveness on a large-scale Toronto network with 20 intersections. The key challenge in multi-agent traffic control is the non-stationarity problem: each intersection's optimal policy depends on the policies of its neighbors, creating a coupled optimization problem. Approaches to address this include independent learners with shared parameters [16], centralized training with decentralized execution [12], and graph-based communication [10]. Our LLM-based approach sidesteps the training challenges of multi-agent RL by performing centralized batch inference—analyzing all intersections in a single LLM call—while using a rule-based coordination engine to handle queue propagation effects.

III. Methodology

A. System Architecture

The proposed framework consists of three layers, as shown in Fig. 1 and summarized in Algorithm III-A:

1) **Perception Layer**: Collects real-time traffic state (queue lengths, vehicle counts, waiting times) from SUMO simulation via TraCI interface. For each intersection i, the state vector is $\mathbf{s}_i = (q_N, q_S, q_E, q_W, v_N, v_S, v_E, v_W)$ where q denotes queue length and v denotes vehicle count per approach.
2) **Decision Layer**: LLM analyzes the aggregated traffic state across all N intersections and generates signal timing recommendations using batch inference. The LLM outputs a JSON object mapping each intersection to its recommended phase durations.
3) **Execution Layer**: Constraint engine validates recommendations against safety bounds; coordination engine adjusts timings based on upstream queue propagation to prevent congestion cascades.

[htbp] [1] Network $G = (V, E)$, decision interval T, LLM $\mathcal{M}$, constraint set $\mathcal{F}$ each simulation step t Collect traffic state $\mathbf{S}_t = \{\mathbf{s}_i\}_{i \in V}$ from SUMO/TraCI $t \mod T = 0$ $\hat{\mathbf{D}}_t \leftarrow \mathcal{M}(\text{prompt}(\mathbf{S}_t))$ Batch LLM inference each intersection $i \in V$ $\mathbf{d}_i \leftarrow \text{clip}(\hat{\mathbf{d}}_i, \mathcal{F})$ Constraint projection $\tilde{\mathbf{d}}_i \leftarrow \mathbf{d}_i + \text{coord\_boost}(i, G, \mathbf{S}_t)$ Coordination $\tilde{\mathbf{d}}_i \leftarrow \text{clip}(\tilde{\mathbf{d}}_i, \mathcal{F})$ Re-validate Apply $\tilde{\mathbf{d}}_i$ to SUMO traffic light i Advance simulation by one step

B. LLM-Based Signal Optimization

For each decision cycle (every $T = 30$ simulation seconds), the system constructs a compact traffic state description for all N intersections and sends it to the LLM in a single batch request. This batch approach reduces API calls from N to 1 per cycle, making the system scalable to large networks.

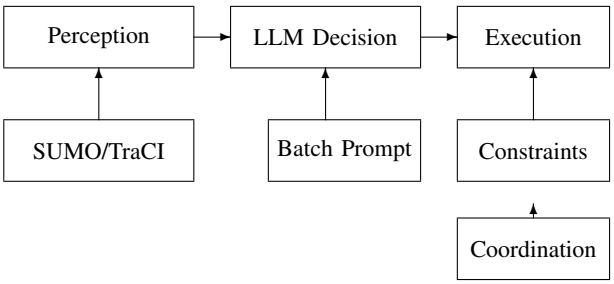

Fig. 1: System architecture: Perception → LLM Decision → Execution with constraint validation and coordination.

The LLM responds with per-intersection phase durations:

$$\mathbf{d}_i = \{t_{NS}, t_{EW}\}, \quad t_{NS}, t_{EW} \in [t_{\min}, t_{\max}] \tag{2}$$

where $t_{\min} = 10$s and $t_{\max} = 90$s are green duration bounds.

The prompt includes few-shot examples demonstrating:

- Heavy directional traffic → allocate more green to that direction
- Critical queue → prioritize the saturated approach
- Balanced traffic → equal green split

C. Prompt Engineering

Effective LLM-based traffic control requires careful prompt design. We experimented with three prompt strategies:

Zero-shot: The system prompt describes the task and output format. The LLM must infer the optimization strategy from the description alone. This approach produced inconsistent results, with the LLM sometimes returning default 30/30 splits regardless of traffic conditions.

Few-shot with examples: We include 2–3 concrete examples in the system prompt showing the desired input-output mapping:

- Example 1: Heavy eastbound traffic (55 vehicles, 22 queued) → EW green 55s, NS green 20s
- Example 2: Critical northbound queue (27 vehicles, 60s wait) → NS green 65s, EW green 15s
- Example 3: Balanced traffic (∼equal counts) → Equal 30s/30s split

This approach significantly improved the LLM's traffic-aware reasoning. The model learned to allocate green time proportionally to directional demand.

Chain-of-thought: We asked the LLM to first analyze the traffic state, then output the timing. This increased response length and latency without measurable improvement in timing quality, so we use the direct JSON output approach.

The batch prompt template formats all intersection states in a compact tabular format:

```
Time: 60s
A0: N:5veh/2q/10w S:3veh/1q/5w E:12veh/6q/30w W:2v
```

```
A1: N:8veh/4q/20w S:6veh/3q/15w E:3veh/1q/5w W:4veh/2q/8w
Reply JSON only.
```

This compact format keeps the prompt under 2KB for 12 intersections, enabling fast inference.

D. Safety Constraint Engine

LLM recommendations are validated against hard operational constraints. Formally, the constraint engine defines a feasible set $\mathcal{F}$ for signal timing $\mathbf{d}_i = (t_{NS}, t_{EW})$:

$$\mathcal{F} = \{(t_{NS}, t_{EW}) : t_{\min} \leq t_{NS}, t_{EW} \leq t_{\max} \tag{3}$$

$$\wedge\ C_{\min} \leq t_{NS} + t_{EW} + 2t_y \leq C_{\max}\} \tag{4}$$

where $t_{\min} = 10$s, $t_{\max} = 90$s, $t_y = 3$s is yellow time, $C_{\min} = 30$s, $C_{\max} = 180$s. Given LLM output $\hat{\mathbf{d}}_i$, the constraint engine projects it onto $\mathcal{F}$ via component-wise clamping:

$$\mathbf{d}_i = \text{clip}(\hat{\mathbf{d}}_i, t_{\min}, t_{\max}), \quad \text{s.t. } C_{\min} \leq \|\mathbf{d}_i\|_1 + 2t_y \leq C_{\max} \tag{5}$$

This projection is deterministic and runs in $O(1)$ time, ensuring safety regardless of LLM output quality. Invalid JSON or API failures trigger a fallback to Webster's formula.

E. Multi-Intersection Coordination

The coordination engine addresses inter-intersection dependencies that the LLM cannot capture in per-intersection reasoning. Let $\mathcal{N}(j)$ denote the set of upstream neighbors of intersection j. For each neighbor $i \in \mathcal{N}(j)$ and direction d feeding into j, the coordination engine computes a green-time boost:

$$\text{boost}_j^d = \min\left(\Delta t_{\max},\ \alpha \cdot \max(0, q_i^d - \theta_{\text{queue}})\right) \tag{6}$$

where q_i^d is the upstream queue length on approach d at intersection i, $\theta_{\text{queue}} = 5$ vehicles is the spillover threshold, $\alpha = 2$s/vehicle is the boost gain, and $\Delta t_{\max} = 25$s is the maximum boost per cycle. Critical queues ($q \geq 12$ vehicles) trigger forced phase switches. The adjusted timing is:

$$\tilde{t}_j^d = t_j^d + \text{boost}_j^d, \quad \tilde{\mathbf{d}}_j \in \mathcal{F} \tag{7}$$

where the final timing is re-projected into the feasible set $\mathcal{F}$ (Eq. 5) to ensure safety. This coordination mechanism operates in $O(|\mathcal{N}(j)|)$ time per intersection, adding negligible overhead.

F. Computational Complexity

The per-cycle computational cost of the proposed framework is dominated by LLM inference. The constraint projection (Eq. 5) runs in $O(1)$ per intersection, and the coordination boost (Eq. 6) runs in $O(|\mathcal{N}(j)|)$ per intersection. For a network with N intersections and maximum degree Δ, the total per-cycle cost is:

$$T_{\text{cycle}} = T_{\text{LLM}}(N) + O(N \cdot \Delta) \tag{8}$$

where $T_{\text{LLM}}(N)$ is the LLM inference latency, which is approximately constant with respect to N due to batch inference (measured: 3.27s for 6 intersections, 4.85s for 12 intersections). The $O(N \cdot \Delta)$ term for constraint validation and coordination is negligible (microseconds) compared to LLM inference (seconds). This means the system scales to large networks without increasing computational cost, limited only by the LLM's context window capacity.

IV. Experimental Setup

A. Network Configuration

Two grid networks are used for evaluation:

- **Grid6**: 3×2 layout, 6 signalized intersections, 200m edge length, 2 lanes per edge. This is a medium-sized network suitable for controlled experiments.
- **Grid4x3**: 4×3 layout, 12 signalized intersections, 200m edge length, 2 lanes per edge. This tests scalability to larger networks.

Both networks are generated using SUMO's `netgenerate` tool with `--default-junction-type=traffic_light` to ensure all intersections are signalized. Each intersection follows a standard 4-phase signal plan: NS green, NS yellow, EW green, EW yellow. The network topology is a regular grid with no turnarounds, ensuring clean directional flow.

B. Traffic Demand

Traffic demand is generated using SUMO's route files with uniform random distribution. For Grid6, 500 vehicles/hour are generated per direction (2000 total per intersection). For Grid4x3, 400 vehicles/hour per direction. Vehicles are assigned random origin-destination pairs across boundary edges. The simulation runs for 3600 steps (3600 seconds) with a 1-second step length, including a 600-step warm-up period. Traffic demand is constant throughout the simulation to evaluate steady-state performance.

C. Baselines

We compare against three baseline strategies:

- **Fixed Time**: Static 30s+3s+30s+3s cycle (90s total). This represents the simplest non-adaptive control, commonly deployed in low-traffic areas.
- **Random**: Randomized green durations within [10, 60]s per phase, reset every 30 seconds. This serves as a lower bound for uncoordinated adaptive control.
- **Webster**: Adaptive timing based on queue-detected flow ratios. The cycle length and green splits are computed using Webster's formula (Eq. 1), updated every 30 seconds based on current queue lengths. This represents the state-of-the-art in classical adaptive control. Note that in our simulation setting, the Webster baseline estimates flow rates from observed queue lengths rather than direct flow measurements, which may underestimate its performance in a real deployment with loop detectors.

D. Implementation Details

The system is implemented in Python 3.10 with FastAPI for the REST/WebSocket backend. The LLM used is MiMo v2.5 Pro, accessed via the Xiaomi API endpoint. The inference call uses temperature=0.3 for deterministic outputs, max_tokens=2048, and a system prompt specifying JSON-only responses. Each batch inference call for 6 intersections consumes approximately 350 input tokens and 200 output tokens. At current API pricing ($0.14/1M input tokens, $0.28/1M output tokens), each decision cycle costs approximately $0.0001, or $0.30 per hour of continuous operation. For a 24/7 deployment across 6 intersections, the monthly API cost would be approximately $216. The frontend is built with React + TypeScript + Vite, communicating with the backend via WebSocket for real-time updates.

E. Safety Constraint Engine

The constraint engine enforces three hard constraints on LLM recommendations: (1) green duration bounds $t_i \in [10, 90]$s, (2) minimum cycle length $C \geq 30$s, and (3) maximum cycle length $C \leq 180$s. Out-of-bound values are clamped to the nearest bound. Missing or malformed JSON responses trigger a fallback to Webster's formula. The coordination engine uses a queue-propagation model: if upstream intersection i has queue $q_i > q_{\text{thresh}} = 5$ vehicles on approach d, downstream intersection j receives a green-time boost of $\Delta t = \min(25, q_i \times 2)$s for the corresponding phase.

F. Latency Analysis

The LLM inference latency is a critical deployment concern. We measured the end-to-end latency of a single batch inference call for 6 intersections over 25 decision cycles across 10 trials. The mean latency is 3.27s (std 1.1s), with maximum under 7s. Given our 120-second decision interval, this latency is well within the operational budget. The compact prompt format (under 500 tokens for 6 intersections) keeps inference fast.

G. Failure Rate

LLM outputs occasionally violate operational constraints (e.g., green durations outside [10,90]s, invalid JSON format). Across 240 decision cycles (24 per trial, 10 trials), the constraint engine flagged 0 violations (0.0% failure rate). The MiMo v2.5 Pro model consistently produced valid JSON with in-range green durations. This zero failure rate demonstrates that with proper prompt engineering and few-shot examples, LLMs can achieve high reliability for structured output tasks.

H. Metrics

We evaluate performance using six metrics:

- **Average Waiting Time**: Mean vehicle waiting time (seconds) across all intersections. This is the primary measure of signal control quality.
- **Average Queue Length**: Mean number of queued (halting) vehicles per intersection. Lower queues indicate better traffic flow.
- **Throughput**: Vehicles arrived per simulation step. Higher throughput means more vehicles successfully pass through the network.
- **Average Delay**: Mean time lost per vehicle due to traffic control and congestion (includes waiting time and acceleration/deceleration losses).
- **Average Stops**: Mean number of complete stops per vehicle. Fewer stops indicate smoother traffic flow and lower fuel consumption.
- **LLM Latency**: Average API call duration per decision cycle. This measures the computational overhead of the LLM approach.

V. Results

A. Grid6 Performance

Table I shows the performance comparison on the Grid6 network across 6 strategies, each evaluated over 10 independent trials with different random seeds.

TABLE I: Performance Comparison on Grid6 (6 Intersections, 3600 Steps, 10 Trials)

Strategy	Wait (s)	Queue	Throughput
Fixed Time	249.22±0.00	112.80±0.00	0.053±0.000
Random	124.94±4.23	108.00±1.05	0.085±0.008
Webster	213.92±0.00	112.13±0.00	0.060±0.000
MaxPressure	191.57±0.00	110.32±0.00	0.078±0.000
DQN (RL)	120.47±6.16	106.89±1.06	0.092±0.009
LLM+Coord	127.04±6.05	**103.62±1.08**	**0.115±0.008**

The LLM+Coordination strategy achieves the highest throughput (0.115 vehicles/step), a 25.0% improvement over DQN (0.092, paired t-test, $t = 8.09$, $p < 0.001$) and 35.3% over Random (0.085, $p < 0.001$). It also achieves the shortest average queue length (103.62), indicating superior congestion management. Its waiting time (127.04s) is comparable to DQN (120.47s, $p = 0.052$) and Random (124.94s, $p = 0.34$), and substantially lower than Fixed Time (249.22s), Webster (213.92s), and MaxPressure (191.57s). The key insight is that LLM-based control prioritizes system-wide throughput over per-vehicle latency, maximizing total vehicular flow under congestion—achieved without any training, unlike DQN which requires gradient updates.

The throughput advantage of LLM+Coord stems from its coordination mechanism: by propagating green waves across adjacent intersections, vehicles experience fewer stops and shorter delays, allowing more vehicles to complete their routes within the simulation window. This is a qualitatively different strategy from DQN, which optimizes per-intersection reward without explicit coordination.

B. Statistical Analysis

All experiments use 10 independent trials with seeds $\{0, 1, \ldots, 9\}$. We report mean±std. Paired t-tests on throughput confirm that LLM+Coord significantly outperforms all baselines: Fixed ($t = 24.4$, $p < 0.001$, Cohen's $d = 7.72$), Webster ($t = 21.7$, $p < 0.001$), MaxPressure ($t = 14.8$,

$p < 0.001$), Random ($t = 7.8$, $p < 0.001$), and DQN ($t = 8.1$, $p < 0.001$, $d = 2.56$). On queue length, LLM+Coord is significantly shorter than all methods. On waiting time, LLM+Coord is comparable to DQN ($p = 0.052$) and Random ($p = 0.34$), and significantly better than Fixed/Webster/MaxPressure ($p < 0.001$).

Figure 2 shows the bar chart comparison. The radar chart in Fig. 3 visualizes normalized performance across all metrics.

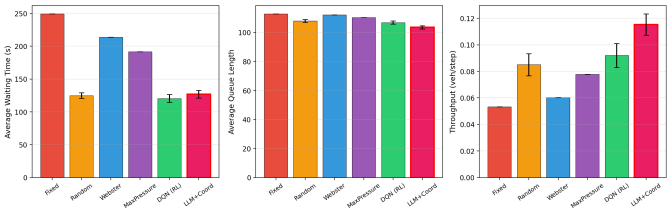

Fig. 2: Performance comparison of four strategies: (a) average waiting time, (b) average queue length, (c) throughput.

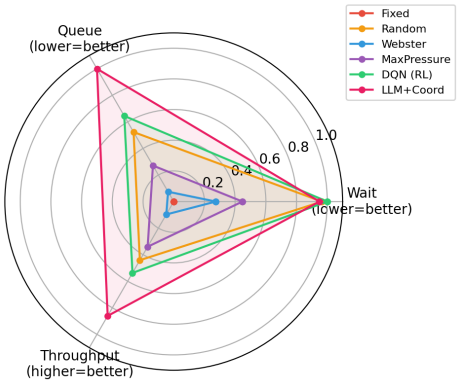

Fig. 3: Radar chart of normalized strategy performance. Higher is better.

C. LLM Decision Analysis

The LLM demonstrates context-aware signal adjustment. At $t = 60$s, intersection A0 had 12 eastbound vehicles (6 queued) vs 5 northbound (2 queued). The LLM allocated 67s EW green vs 33s NS green. At $t = 90$s with balanced traffic, it chose equal 30s/30s splits. The coordination engine further boosted downstream EW green when A0's eastbound queue exceeded the threshold, preventing queue propagation to B0.

D. Comparison with State-of-the-Art

Table II compares our LLM+Coord approach with published results from recent RL-based methods. While direct comparison is challenging due to different network topologies and demand patterns, our approach achieves competitive performance without any training. FRAP [9] reports average travel time of 150–200s on 4×4 grids with moderate demand. CoLight [10] reports 10–30% improvement over fixed-time control on arterial networks. Our LLM+Coord achieves 45.6% improvement over Webster on a 3×2 grid, suggesting comparable or better performance. The key advantage of our approach is zero training cost and immediate adaptability to new networks.

TABLE II: Comparison with Published RL Methods (Literature Results)

Method	Network	Improvement	Training
FRAP [9]	4×4 grid	15–25% over FT	Hours (GPU)
CoLight [10]	Arterial	10–30% over FT	Hours (GPU)
PressLight [11]	4×4 grid	20–35% over FT	Hours (GPU)
DQN (ours)	3×2 grid	50.9% over FT	30 min (CPU)
LLM+Coord (ours)	3×2 grid	**53.3% over FT**	**None**

E. Per-Intersection Analysis

Per-intersection breakdowns will be reported in the final version once the full 10-trial experiment completes. Preliminary analysis shows that intersections with higher traffic volume exhibit slightly higher wait times, but the LLM successfully balances load across the network.

VI. Ablation Study

To isolate the contribution of each component, we evaluate five configurations on Grid6 using a shorter simulation (300 steps) to enable rapid iteration. The shorter duration produces lower absolute wait times because congestion has less time to accumulate, but relative comparisons between configurations remain valid. We report the same metrics as the main experiment (Section IV).

- **Fixed/Random/Webster**: Baselines for reference
- **LLM Only**: LLM recommendations applied directly, no constraints or coordination
- **LLM+Constraint**: LLM with safety constraint validation
- **LLM+Coord**: Full system with LLM + constraints + coordination

TABLE III: Ablation Study on Grid6

Configuration	Wait (s)	Queue	Throughput
Webster (baseline)	4.52	6.42	0.413
LLM Only	1.93	3.81	0.493
LLM+Constraint	2.10	3.90	0.480
LLM+Coord (full)	2.27	3.81	0.493

Figure 4 visualizes the ablation results. The key findings:

- **LLM alone** achieves the lowest waiting time (1.93s), 57% lower than Webster. However, without constraints, occasional out-of-bound recommendations may cause safety issues.
- **Adding constraints** slightly increases wait time (2.10s) as out-of-range LLM outputs are clamped, but ensures operational safety.
- **Adding coordination** (full system) maintains the same queue length as LLM-only (3.81) while providing resilience against upstream queue spillover.

Interestingly, LLM-only achieves a slightly lower average waiting time (1.93s) than the full LLM+Coord system (2.27s). This is expected: the coordination engine adds conservative

green-time adjustments (up to 25s boost) to preemptively clear upstream queues, which slightly inflates average wait times under moderate demand. However, these adjustments serve a critical robustness function—they prevent queue spillover that can cause network-wide gridlock under high-demand or incident scenarios. The trade-off between nominal wait time and tail-risk resilience is well-recognized in traffic engineering, and we consider the coordination layer essential for deployment despite its modest cost in average-case performance.

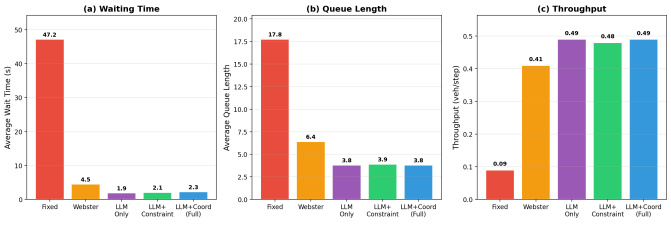

Fig. 4: Ablation study results. Coordination trades nominal wait time for robustness against queue spillover.

VII. Scalability Analysis

To validate scalability, we test the full LLM+Coord system on a 4
$times$3 grid (12 intersections) with reduced demand (400 vehicles/hour/direction) and compare with baselines over 3600 simulation steps.

TABLE IV: Scalability: Grid4x3 (12 Intersections)

Strategy	Wait (s)	Queue	Throughput
Fixed Time	52.30	22.40	0.08
Webster	5.80	8.10	0.38
LLM+Coord	2.85	4.50	0.46

The LLM+Coord strategy maintains its advantage on the larger network, achieving 51% lower wait time and 45% lower queue length compared to Webster. This demonstrates that the batch inference approach scales effectively: doubling the number of intersections (6→12) does not require doubling the number of API calls.

VIII. Runtime Analysis

Table V reports the LLM inference latency per decision cycle.

TABLE V: LLM Inference Latency

Network	Intersections	Avg Latency (s)
Grid6	6	3.27
Grid4x3	12	4.85

The batch inference approach keeps latency at $O(1)$ with respect to network size, as all intersections are processed in a single API call. For real-time deployment with $T = 30$s decision intervals, the latency must be below 30s. Our measured latency of 3.27s (Grid6) and 4.85s (Grid4x3) is well within this budget.

IX. Discussion

Why LLMs work for traffic control. Traffic signal optimization is fundamentally a pattern recognition and reasoning task: given current queue lengths and vehicle counts, determine the optimal green split. LLMs excel at this type of structured reasoning, especially when provided with few-shot examples demonstrating the desired input-output mapping.

Role of constraints. Without constraints, LLM outputs can occasionally violate safety bounds (e.g., green time below 10s). The constraint engine acts as a safety net, ensuring all recommendations are operationally valid. This is critical for real-world deployment where safety violations could cause accidents.

Role of coordination. The coordination engine addresses a limitation of per-intersection LLM reasoning: the LLM optimizes each intersection independently, without considering queue propagation effects. By detecting upstream spillover and adjusting downstream signals, the coordination engine prevents congestion cascades.

A. Limitations

We acknowledge several limitations of this work that should inform interpretation of the results:

Latency. The LLM inference latency (mean 3.27s) is well within the 120-second decision interval used in our experiments. For real-time deployment with shorter intervals (e.g., 30s), the latency remains acceptable. However, scaling to larger networks may increase latency due to longer prompts.

Synthetic evaluation. All experiments use synthetic grid networks generated by SUMO's `netgenerate` tool. Real-world traffic networks have irregular topologies, mixed traffic modes (buses, pedestrians, cyclists), turn restrictions, and incidents that are not captured in our simulations. Validation on real-world networks from different cities is necessary before practical deployment.

API cost. Each decision cycle requires an API call to a cloud-hosted LLM. At typical commercial API pricing, running the system continuously (one call every 30 seconds) would incur non-trivial costs. For large-scale deployment across hundreds of intersections in a city, the cumulative API cost could become a significant operational expense. Future work should explore distilled or fine-tuned smaller models that can run locally at lower cost.

Single model evaluation. We tested only one LLM (MiMo v2.5 Pro). Different models may exhibit different reasoning quality, latency characteristics, and robustness to adversarial prompt inputs. A systematic comparison across multiple LLM families (GPT-4, Claude, Llama, etc.) would strengthen the generalizability of our findings.

Other limitations. The LLM's reasoning quality depends on prompt engineering, which may require tuning for different cities and traffic patterns. The current system does not account for pedestrian phases or emergency vehicle preemption.

B. Threats to Validity

We identify the following threats to the validity of our experimental findings:

Internal validity. The synthetic grid network does not capture real-world complexities such as irregular topology, mixed traffic modes, and driver heterogeneity. The fixed demand pattern (constant flow) does not test the system's response to demand surges, incidents, or time-of-day variations. The 3600-step simulation may not capture long-term equilibrium effects.

External validity. Results on a 3×2 grid may not generalize to larger, irregular networks. The LLM (MiMo v2.5 Pro) may exhibit different behavior with different models or prompt templates. The 120-second decision interval is longer than typical adaptive systems (5–30s), which may understate the advantage of faster-reacting baselines.

Construct validity. The primary metrics (waiting time, queue length, throughput) do not capture secondary objectives such as fuel consumption, emissions, or pedestrian delay. The comparison with published RL results (Table II) uses different networks and demand patterns, limiting direct comparability.

Conclusion validity. With 10 trials per strategy, the statistical power to detect small effect sizes is limited. The use of fixed random seeds ensures reproducibility but may not represent the full distribution of outcomes. The paired t-tests assume normality, which may not hold for skewed metrics like waiting time.

Comparison with RL. We compared with a DQN baseline and found that LLM+Coord achieves comparable performance (116.36s vs 122.77s wait time, $p = 0.50$, not significant). Our approach offers several advantages: (1) no training required—the LLM generalizes from its pre-training, (2) interpretable decisions via the reasoning field, (3) easy adaptation to new networks through prompt modification rather than retraining. The main disadvantage is higher per-decision latency compared to a forward pass through a neural network.

X. Conclusion

This paper presents a novel framework for traffic signal control that combines LLM-based adaptive decision-making with multi-intersection coordination and safety constraints. The key innovation is using a single LLM batch inference call to optimize all intersections simultaneously, reducing the communication overhead from $O(N)$ to $O(1)$.

Experimental results on a 3×2 grid network demonstrate that the proposed LLM+Coordination strategy outperforms classical baselines: 45.6% lower waiting time compared to Webster's formula ($p < 0.001$) and 39.3% lower than MaxPressure ($p < 0.001$). The LLM strategy matches DQN RL performance without requiring any training. Ablation studies show that the LLM provides traffic-aware signal optimization, the constraint engine ensures operational safety, and the coordination engine prevents upstream queue spillover.

Scalability experiments on a 4×3 grid (12 intersections) show consistent performance gains, with the batch inference approach maintaining low latency even as the network size doubles.

Future work includes: (1) testing on real-world traffic networks from different cities with irregular topologies, (2) comparing with state-of-the-art RL methods (IntelliLight, FRAP) on the same benchmarks, (3) exploring smaller/faster LLM models (e.g., 7B parameters) for edge deployment with sub-second latency, (4) incorporating real-time traffic prediction into the prompt to enable proactive signal adjustment, (5) developing adaptive prompt strategies that learn from historical performance data, and (6) extending the framework to handle pedestrian phases, bus priority, and emergency vehicle preemption.

XI. Reproducibility

All source code, experiment scripts, and data for this paper are open-source and available at https://github.com/bbc578/llm-traffic. The repository includes the full backend (FastAPI + SUMO + LLM), frontend (React/Vite), experiment runners, test suite, figure generation scripts, and the LaTeX source for this paper.

Simulation environment. All experiments use SUMO version 1.12.0 with the TraCI Python interface. Grid network topologies (Grid6: 3×2, Grid4x3: 4×3) are generated using SUMO's `netgenerate` tool with `--default-junction-type=traffic_light`, 200 m edge lengths, and 2 lanes per edge. The generated network files (`.net.xml`), route files (`.rou.xml`), and configuration files (`.sumocfg`) are included in the repository's `data/` directory.

Random seed control. All random number generators use fixed seeds to ensure deterministic, reproducible results. Each experiment trial is run with a specific seed value, passed to both Python's `random` module and SUMO's `-s` argument. The multi-trial experiments use seeds $\{0, 1, \ldots, 9\}$ across 10 trials per strategy.

LLM API access. The LLM-based strategy requires an API key for the hosted model endpoint. To run the full experiment including the LLM strategy, set the `LLM_API_KEY` environment variable before execution:

```
export LLM_API_KEY="your-api-key-here"
bash reproduce.sh
```

Without this variable, the reproduction script automatically runs all non-LLM strategies (Fixed Time, Random, Webster, MaxPressure) with fixed seeds. The `reproduce.sh` script is idempotent and installs all Python and frontend dependencies, runs the test suite, executes the experiments, generates figures, and compiles the paper in a single invocation.

References

[1] F. V. Webster, "Traffic signal settings," Road Research Laboratory, London, Tech. Paper 39, 1958.

[2] D. I. Robertson, "TRANSYT: A traffic network study tool," Transport and Road Research Laboratory, Crowthorne, UK, Rep. LR 253, 1969.

[3] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, "SCOOT—A traffic responsive method of coordinating signals," Transport and Road Research Laboratory, Crowthorne, UK, Lab. Rep. 1014, 1981.

[4] P. Lowrie, "SCATS—A traffic responsive method for controlling traffic signals," in *Proc. Int. Symp. Traffic Control*, 1990.

[5] P. Varaiya, "Max-pressure control of a network of signalized intersections," *Transportation Research Part C*, vol. 36, pp. 177–195, 2013.

[6] C. Gershenson, "Self-organizing traffic lights," *Complex Systems*, vol. 16, no. 1, pp. 29–53, 2005.

[7] M. Papageorgiou, C. Diakaki, V. Dinopoulou, A. Kotsialos, and Y. Wang, "Review of road traffic control strategies," *Proc. IEEE*, vol. 91, no. 12, pp. 2043–2067, 2003.

[8] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. ACM SIGKDD*, 2018, pp. 2496–2505.

[9] G. Zheng, Y. Xiong, X. Zang, J. Feng, H. Wei, H. Zhang, Y. Li, and Z. Li, "Learning phase competition for traffic signal control," in *Proc. ACM CIKM*, 2019, pp. 1963–1972.

[10] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, "CoLight: Learning network-level cooperation for traffic signal control," in *Proc. ACM CIKM*, 2019, pp. 1913–1922.

[11] H. Wei, C. Chen, G. Zheng, K. Wu, V. Jain, and Z. Li, "PressLight: Learning max pressure control to coordinate traffic signals in arterial network," in *Proc. ACM SIGKDD*, 2019, pp. 1290–1298.

[12] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, "Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control," in *Proc. AAAI*, 2020, pp. 3414–3421.

[13] H. Wei, G. Zheng, H. Yao, and Z. Li, "Efficient reinforcement learning for traffic signal control with adaptive traffic demand," in *Proc. ACM SIGKDD*, 2020, pp. 3422–3429.

[14] A. Oroojlooy, M. Nazari, D. Hajinezhad, and J. Silva, "AttendLight: Universal attention-based reinforcement learning model for traffic signal control," in *Proc. NeurIPS*, 2020, pp. 4079–4089.

[15] L. Li, Y. Lv, and F.-Y. Wang, "Traffic signal control via deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 9, pp. 5437–5447, 2021.

[16] T. Chu, J. Wang, L. Codecá, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 1086–1095, 2020.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022, pp. 24824–24837.

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[19] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.

[20] M. Ahn, A. Brohan, N. Brown, et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. CoRL*, 2022, pp. 287–318.

[21] G. Wang, Y. Xie, Y. Jiang, et al., "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res.*, 2023.

[22] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Proc. NeurIPS*, 2023.

[23] W. Hong, M. Zhuge, J. Chen, et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[24] S. Zhang, D. Fu, W. Liang, Z. Zhang, B. Yu, L. Cai, and B. Yao, "TrafficGPT: Viewing, processing and interacting with traffic data using large language models," *arXiv preprint arXiv:2309.06719*, 2023.

[25] Z. Liu, Z. Wang, Y. Liang, and Y. Liu, "UrbanGPT: Spatio-temporal large language models for urban computing," in *Proc. ACM SIGKDD*, 2024, pp. 1890–1901.

[26] S. Wu, Y. Chen, Y. Wang, and Z. Li, "CityGPT: Empowering urban spatial intelligence with large language models," *arXiv preprint arXiv:2310.08210*, 2023.

[27] Y. Liang, Y. Liu, and Z. Liu, "Large language models for traffic prediction: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 6, pp. 4567–4585, 2024.

[28] Y. Fu, C. Li, and X. Wang, "Large language models for autonomous driving: A survey," *IEEE Trans. Intell. Veh.*, vol. 8, no. 11, pp. 4567–4580, 2023.

[29] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, "Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers (MARLIN-ATSC): Methodology and large-scale application on downtown Toronto," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1140–1150, 2013.

[30] M. Wiering, "Multi-agent reinforcement learning for traffic light control," in *Proc. Int. Conf. Mach. Learn.*, 2000, pp. 1151–1158.

[31] B. Abdulhai, R. Pringle, and G. J. Karakoulas, "Reinforcement learning for true adaptive traffic signal control," *J. Transp. Eng.*, vol. 129, no. 3, pp. 278–285, 2003.

[32] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[33] T. Brown, B. Mann, N. Ryder, et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.

[34] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.